

INF201 – Lecture 04

28th Sept 2026

Today's topics

- Brief recap
- GenAI in INF201
- Mandatory exercises
- Version control
- Git (and GitHub)
- Common file formats

Brief recap for new (and old) students

- We use Python and Visual Studio Code (VSC)
- We work in virtual environments
- We write well-formatted code
 - Follow [PEP8](#) style guide
 - Functions have doc-strings
 - Functions have type-hints
- We have discussed how Python is a dynamically typed language
- We have done a deep dive into lists
 - Copying lists (including deepcopy)
 - List comprehension
 - List unpacking
- We learned how to work with files and directories in Python using the pathlib library
- This is an advanced course, so I am assuming previous Python knowledge

How do we use GenAI in INF201?

- Not allowed: Copy the entire assignment into a GenAI and just hand in whatever it gives you
- Other use of GenAI for assignments is allowed, but we want to see your AI chat log
 - For summarising a long log, prompt: “Could you list the questions I have asked you and give a short summary of our conversation?”
- For now, we use GenAI as an advanced search engine, we will look at agentic coding later
- Examples of good use:
 - «Could you give me an overview of basic Python that I can review before my next course?»
 - «I don’t understand what this specific line of code does, could you explain it? Could you give me a more beginner friendly alternative»
 - «I have a file of <type>, how can I read this in Python?»
 - «I am getting this error message, what could it mean?»
 - «I am stuck on this assignment. Without solving it for me, could you give me a hint on how to start?»

Mandatory assignments (week 37- 40)

- This is the last week of weekly exercises with mandatory attendance
- Since we have a group of new students joining the course late, there will be an extra assignment in week 41
 - You can also join this one if you are missing some points from the first four weeks
 - This is in addition to the extra assignment that will be given at the end of the term
- If you have at least 13 points from these first weeks, you are good
- The rest of term will have mandatory deliverables every other week
 - Attendance in groups for these weeks is not mandatory but absolutely recommended!

"FINAL".doc



FINAL.doc!



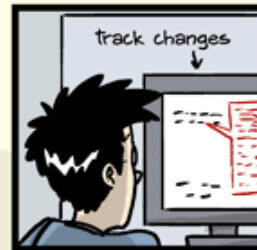
FINAL_rev.2.doc



FINAL_rev.6.COMMENTS.doc



FINAL_rev.8.comments5.
CORRECTIONS.doc



FINAL_rev.18.comments7.
corrections9.MORE.30.doc



FINAL_rev.22.comments49.
corrections.10.#@\$%WHYDID
ICOMETOGRADSCHOOL?????.doc

JORGE CHAM © 2012

Why use version control

- Keep track of changing code
- Test new ideas without breaking working code
- Provide alternative versions of code
- Localize where bugs were introduced
- Collaborate with others
- Document contributions

"FINAL".doc



FINAL.doc!



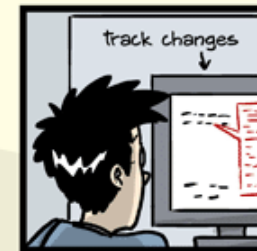
FINAL_rev.2.doc



FINAL_rev.6.COMMENTS.doc



FINAL_rev.8.comments5.
CORRECTIONS.doc



FINAL_rev.18.comments7.
corrections9.MORE.30.doc



FINAL_rev.22.comments49.
corrections.10.#@\$%WHYDID
ICOMETOGRADSCHOOL?????.doc



JORGE CHAM © 2012

How to use version control



Git

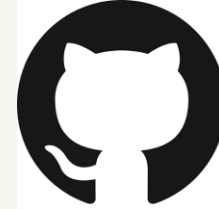
- The by far most used version control system
- Download git here: [Git – Install](#)
- Git runs on your computer (local)
- You interact with git to register changes in your work
- This is done either on command line or directly in Visual Studio Code

How to use version control



Git

- The by far most used version control system
- Download git here: [Git – Install](#)
- Git runs on your computer (local)
- You interact with git to register changes in your work
- This is done either on command line or directly in Visual Studio Code



GitHub

- Server supporting git repositories
- Off-site back-up
- Collaborating with others
- Other options: GitLab, Atlassian (BitBucket)
- GitHub is owned by Microsoft and works well with VS Code
- [Start your journey - GitHub Docs](#)
- [Quickstart: use source control in VS Code](#)

Git Resources

- [ProGit](#) by Scott Chacon and Ben Straub
 - Free book about everything git
 - From 2014, so some parts are a bit outdated
- [Quickstart: use source control in VS Code](#)
 - Guide for using git in VS Code
 - Guide for setting up your first git repository
 - Covers linking up with GitHub

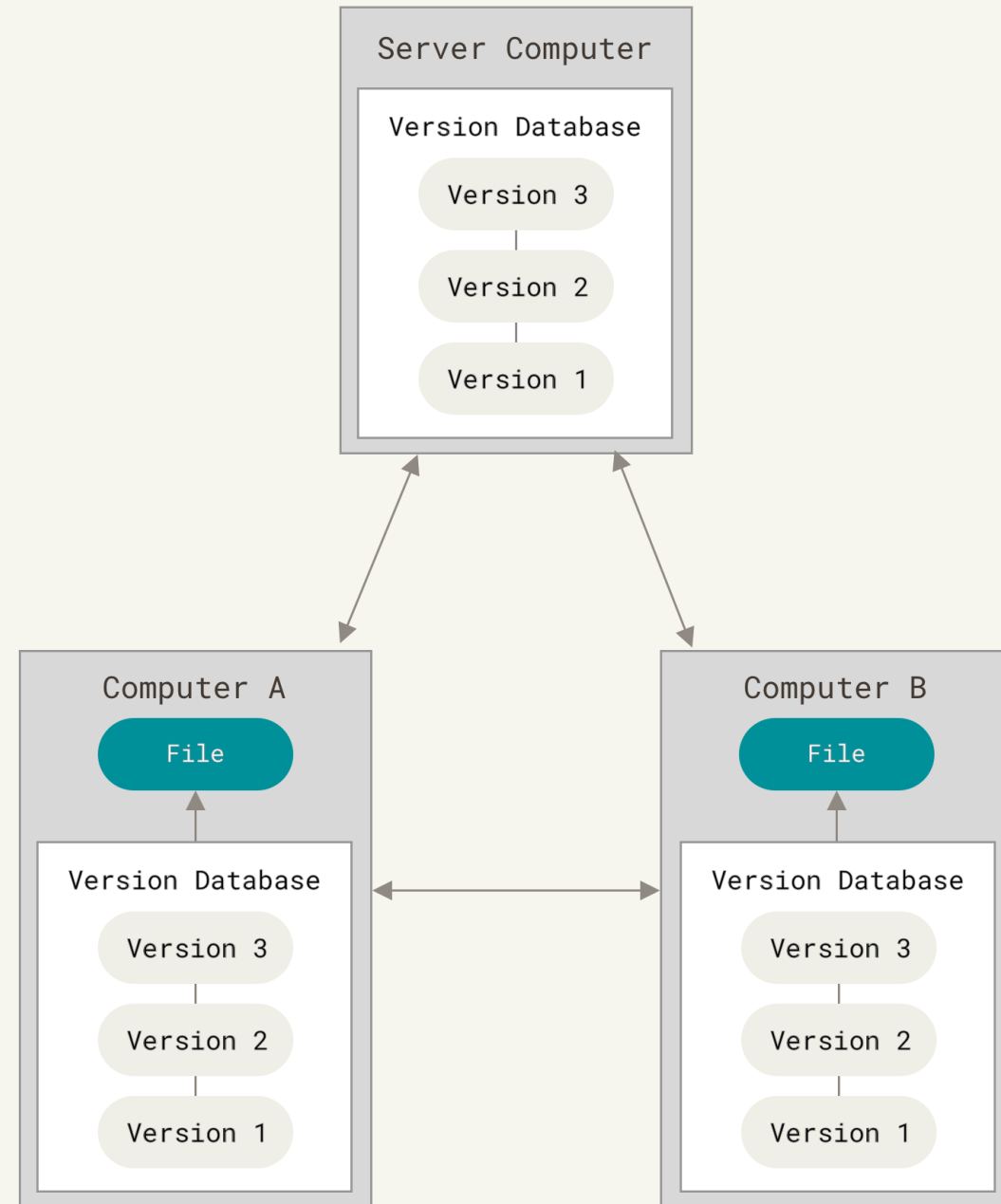
- [Learn Git Branching](#)
 - Really good visual guide to git commands
- [Lecture 5: Version Control and Git](#)
 - Lecture on Git by [The Missing Semester of Your CS Education](#) at MIT
 - Excellent overview on how git manages data

[Introduction to Git - Training | Microsoft Learn](#)



How does git work?

- Git is an example of distributed version control
 - The complete history of your project is stored locally on your computer
 - Supports communication with remote servers
 - If you use a server like GitHub, the complete history is available there after you sync with the server
 - You can *push* your changes to the server
 - You can *pull* changes from the server
 - You can *merge* changes from several contributors



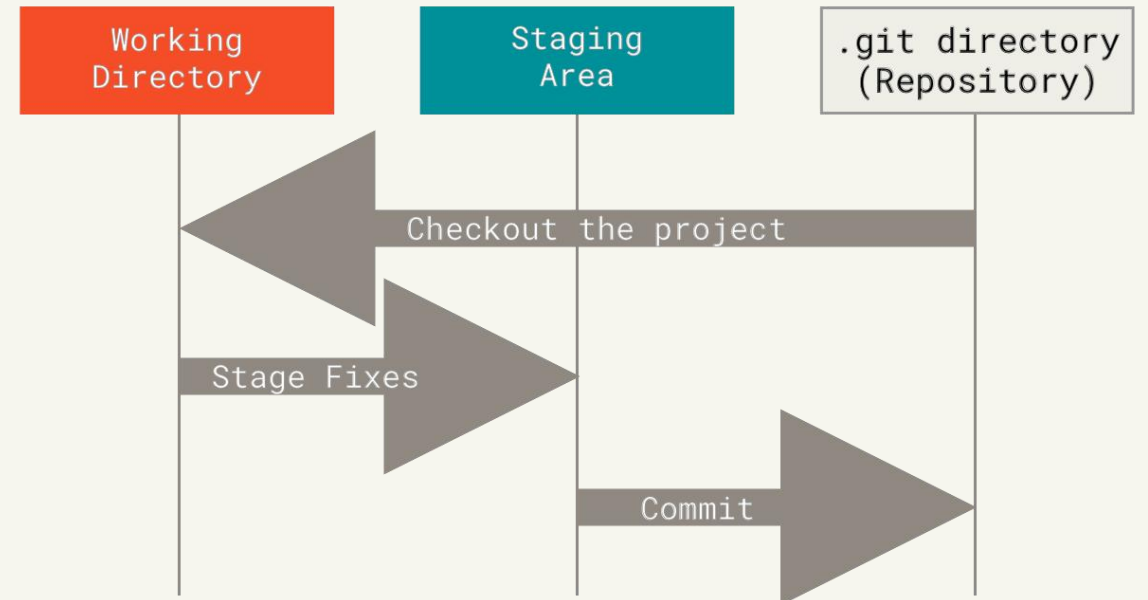
Git terminology

- Repository
 - A collection of folders and files managed by Git
 - Git stores all information about history in a special folder `.git` (hidden folder)
 - The `.git` folder is sometimes also called the repository
 - Do not touch the `.git` folder!
- Working directory
 - Folder with files you work on
- Cloning
 - Copy an entire repository from a server to your computer
 - Usually done once
 - Automatically creates working directory with current version of project
- Staging
 - Tell Git which changes you want to register
- Committing
 - Tell Git to register (check in) changes in repository
 - A commit refers to all changes registered simultaneously
 - Each commit defines a version (also called snapshot)



Basic Git workflow

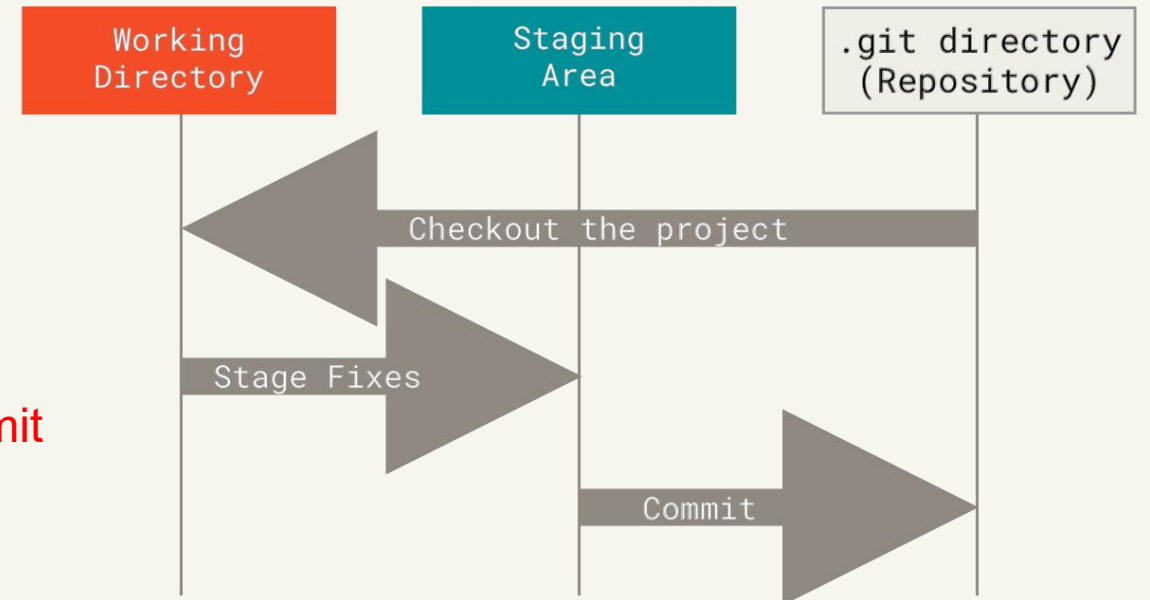
1. Clone repository (only once)
2. Make changes to code
3. Stage changes: decide what to check-in
4. Commit changes: check-in and provide short message on changes made
5. Do this *often* and in *small* increments



[ProGit](#) by Scott Chacon and Ben Straub

Basic Git workflow

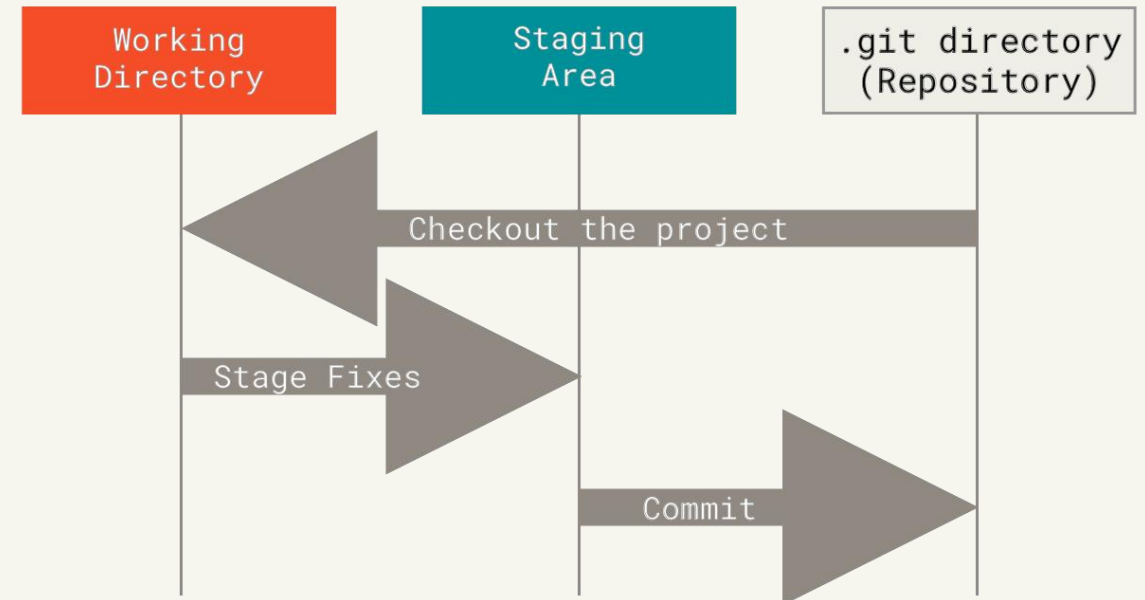
1. Clone repository (only once) `git clone`
2. Make changes to code
3. Stage changes: decide what to check-in `git add`
4. Commit changes: check-in and provide short message on changes made `git commit`
5. Do this *often* and in *small* increments



[ProGit](#) by Scott Chacon and Ben Straub

What's the deal with the staging area?

- *git add* selects specific changes from your working directory and places them into the staging area.
- *git commit* packages whatever is currently inside the staging area into a permanent record in the Git history log.
- Having a separate staging area allows you to curate and inspect exactly which changes should be committed



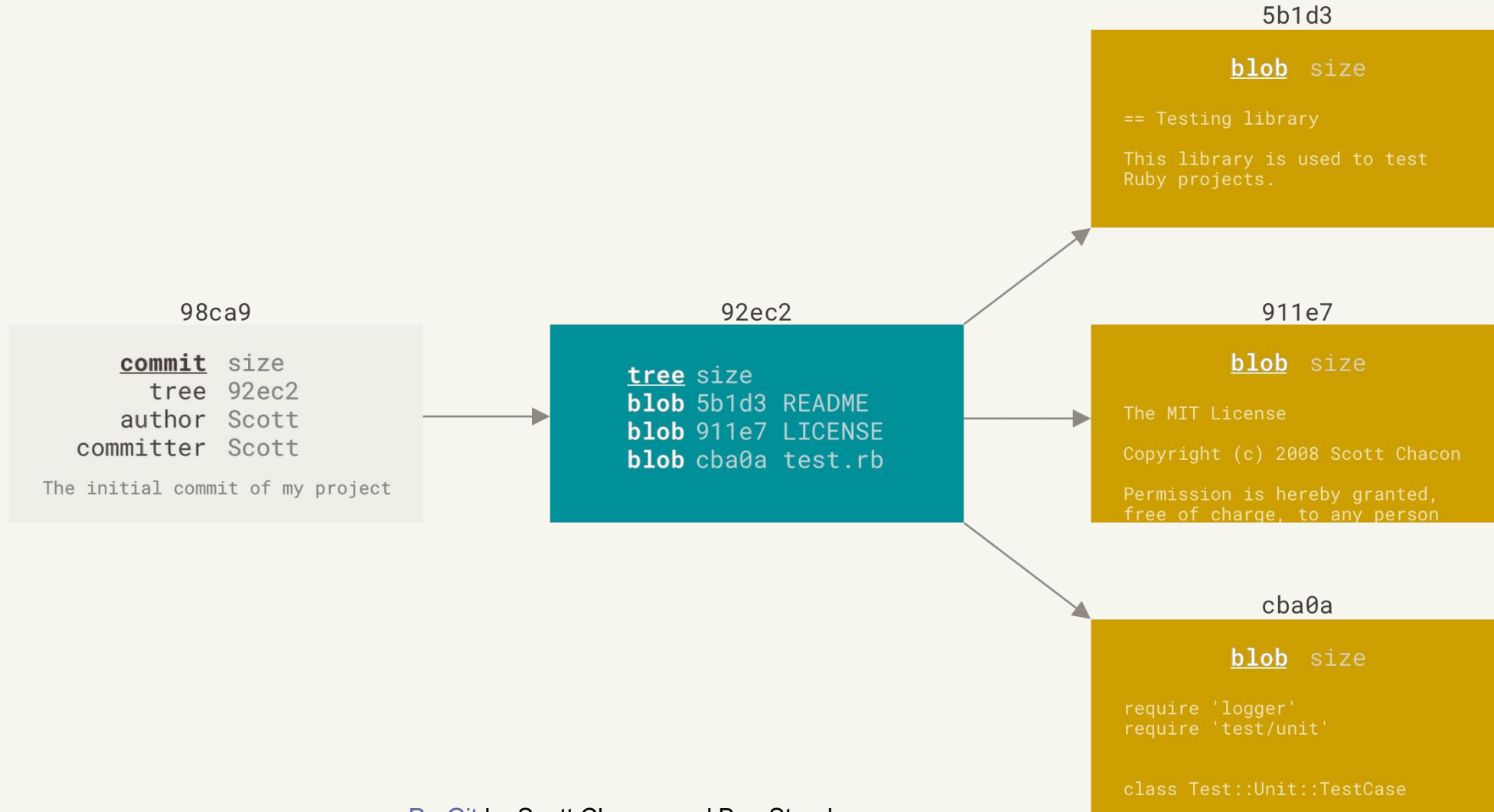
[ProGit](#) by Scott Chacon and Ben Straub

What's the deal with the staging area?

- *git add* selects specific changes from your working directory and places them into the staging area.
- *git commit* packages whatever is currently inside the staging area into a permanent record in the Git history log.
- Having a separate staging area allows you to curate and inspect exactly which changes should be committed
- If you have made several unrelated changes to your code, the staging area allows you to separate them into clean, logical commits
- You can choose to leave experimental code, temporary print statements, or work-in-progress files out of a commit while still committing the clean parts that are ready
- It gives you a checkpoint to review your work before locking it into history
- Helpful during *merge* conflicts



What is a commit?

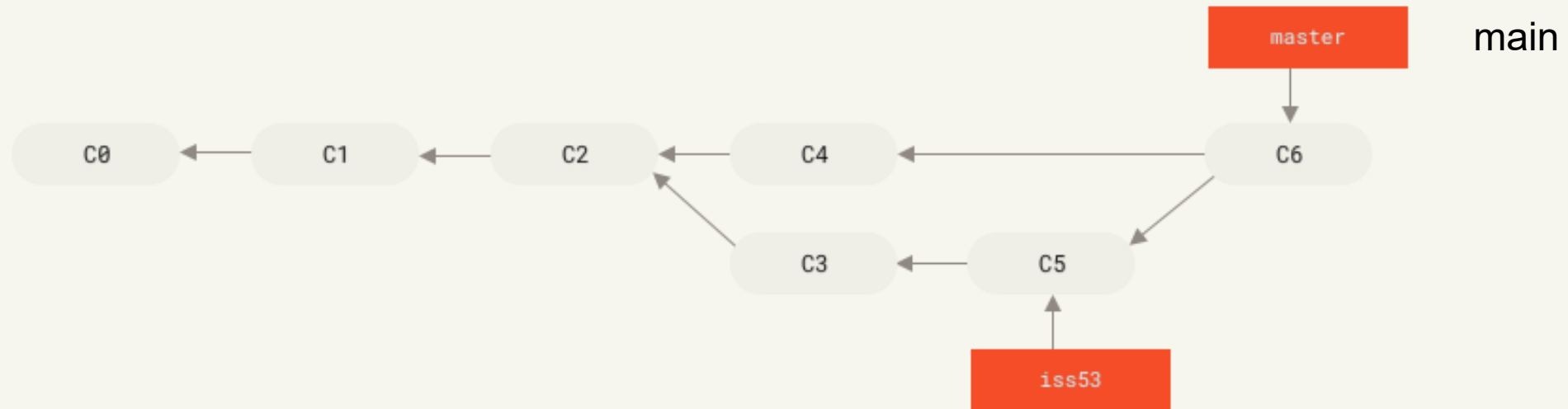


What is a commit? Some more terminology...

- Blob
 - A localized change to a single file
 - Multiple changes to a single file may be separate blobs if they are far enough from each other in the file
- Tree
 - Collection of all blobs in a commit
- Hash
 - A unique digital fingerprint of each blob, tree and commit
- Commit
 - Tree with all changes checked in with this commit
 - Metadata
 - Information about author
 - Explanatory message about content of commit
 - Hash of single parent or both parents (previous commit(s))
 - Hash of commit itself
- Git history as a blockchain
 - By linking via hashes, Git builds a definite history
 - One cannot change earlier commits without breaking the chain



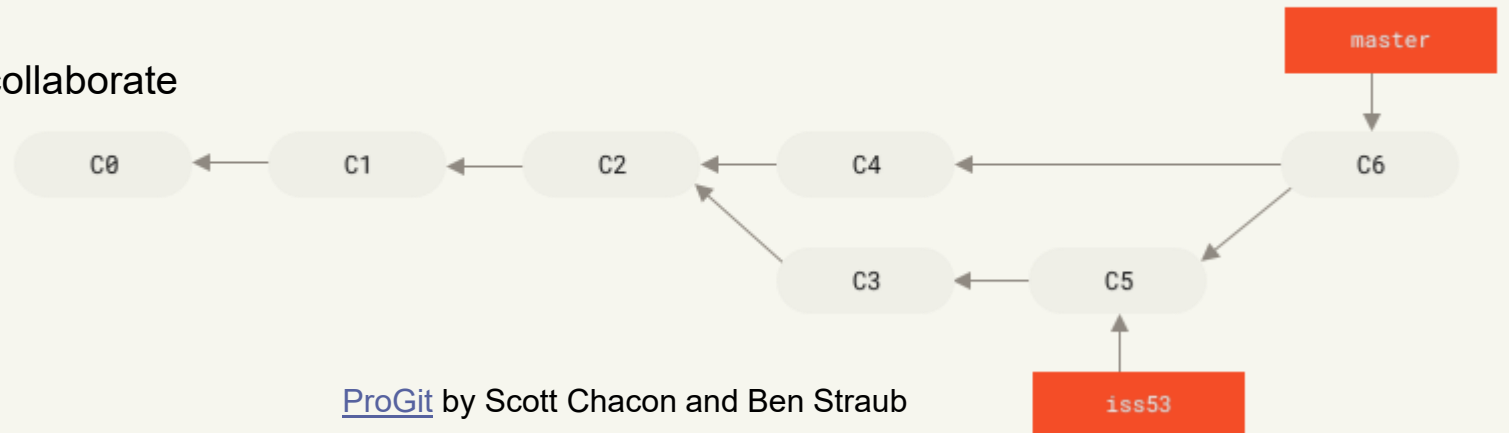
Branches are very useful



[ProGit](#) by Scott Chacon and Ben Straub

Branches are very useful

- Extremely powerful feature to work on code in a controlled way
- Changes made in a branch are kept separate from changes in other branches
 - You can try out new things in a branch
 - You can commit as many versions in a branch as you like
 - If your new ideas work out, you can integrate them
 - If they don't work out, you can just delete the branch
 - You can share branches with others to collaborate
- Special branch for "definite" version of project
 - Historically called master, today usually called main
 - Automatically created when you create a repository
 - Changes from successful branches get integrated into main



More Git terminology

- Branching
 - Create a new branch
 - E.g., to fix a bug or develop a new feature
 - Usually want to ensure all changes are committed before creating a new branch
 - Each branch is a pointer to a specific commit
- Checking out
 - Switching content of working directory between branches
 - Usually want to ensure all changes are committed before checking out a different branch
 - In newer versions of git there is more intuitive command called git switch
- HEAD
 - A pointer to the current branch (“You are here!”)
 - Each new commit is added as a child of the commit HEAD points to
 - Upon commit, the current branch pointer (HEAD) is moved to the new commit



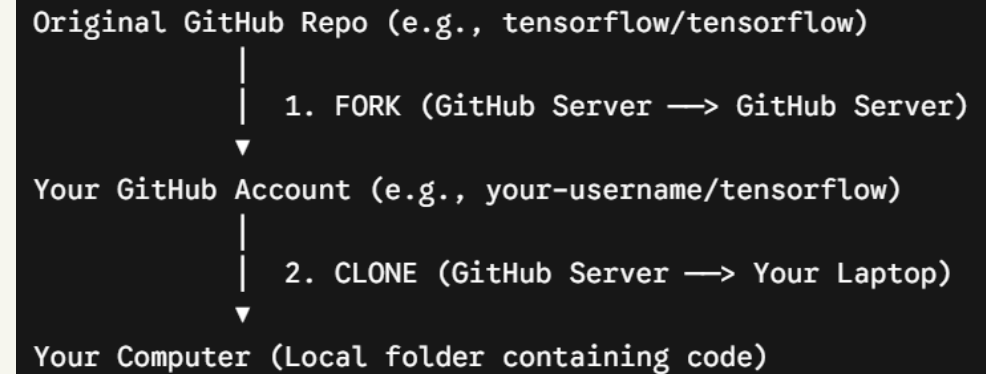
More Git terminology

- Merging
 - Combine changes made on different branches
 - from bug-fix branch into main
 - merge new feature into main
 - merge changes from remote server into your main
 - merge changes from main into a branch to keep branch up-to-date
 - Usually git handles necessary changes automatically
 - If possible, git performs fast forward: changes from merged branch just put at head of current branch
 - Otherwise, creates merge commit with two parents
- Conflicts
 - Sometimes, git cannot combine changes from two branches
 - Usually if different changes to the same line of code
 - Git flags a conflict
 - Programmer must resolve conflict manually and then commit
 - Note: Git can only detect conflicts in program text, not in program logic



More Git terminology – Working with remote server

- Git Server
 - Any computer providing access to git repositories
 - For example GitHub
- Clone
 - Copy repository from server to your machine
- Fork
 - Copy a repository, usually from a different user, on a server
 - For example: Fork from another GitHub repository to your GitHub repository
 - Not relevant for this course, but necessary if you want to contribute to a project



More Git terminology – Working with remote server

- **Git Server**
 - Any computer providing access to git repositories
 - For example GitHub
- **Clone**
 - Copy repository from server to your machine
- **Fork**
 - Copy a repository, usually from a different user, on a server
 - For example: Fork from another GitHub repository to your GitHub repository
 - Not relevant for this course, but necessary if you want to contribute to a project
- **Remote**
 - A remote is a git server that you have told your local git repository about
 - When you clone a project from a server, that server is automatically registered as the remote origin
- **Fetch**
 - Get information about all new changes available at a remote
 - Does not change your working directory
- **Pull**
 - Merge changes from remote branch into your working directory
- **Push**
 - Send your commits to remote server
 - Will fail if server has commits not yet in your history
 - Need to pull first



Git commands – An overview

1. Setup and identity
2. Getting started
3. Daily local use
4. Syncing with remote (GitHub)
5. Inspecting changes
6. Branching
7. Safety and diagnosis

Git commands – An overview

1. Setup and identity
2. Getting started
3. Daily local use
4. Syncing with remote (GitHub)
5. Inspecting changes
6. Branching
7. Safety and diagnosis

The 5-command starter pack

- `git clone`
- `git status`
- `git add`
- `git commit -m "..."`
- `git push`

Git commands – Setup and identity

Run once per machine

- `git config --global user.name "Your Name"`
 - Sets the author name attached to your commits.
- `git config --global user.email "email@example.com"`
 - Links local commits to your GitHub profile (must match your GitHub email).
- `git config --global init.defaultBranch main`
 - Ensures new local repositories start with main as the default branch name.



Git commands – Getting started

- `git clone <url>`
 - Downloads a repository from GitHub to your local machine.
- `git init`
 - Initializes a brand-new, empty Git repository in your current folder.

Git commands – Local daily use

- `git status`
 - Displays modified files, staged changes, and untracked files.
- `git add <file>` (or `git add .`)
 - Moves modified files into the staging area.
- `git commit -m "Descriptive message"`
 - Saves staged changes as a permanent snapshot in your local repository log.



Git commands – Syncing with remote (GitHub)

- `git push`
 - Uploads local commits to GitHub.
- `git pull`
 - Downloads and integrates changes from GitHub into your local branch.
 - Combines `git fetch` (fetching new commits) and `git merge` (merge them into current branch)



Git commands – Inspecting changes

- `git log --oneline`
 - Displays a simplified, clean timeline of past commits.
- `git diff`
 - Shows line-by-line differences between your current code and the last commit.



Git commands – Branching

- `git switch -c <branch-name>`
 - Creates a new branch and immediately switches to it.
 - More recently introduced command, use instead of `git checkout`
- `git switch <branch-name>`
 - Switches back and forth between existing branches (e.g., `git switch main`).
- `git merge <branch-name>`
 - Combines changes from the specified branch into your current active branch.



Git commands – Safety and diagnostics

- `git restore <file>`
 - Discards uncommitted local changes in a file, reverting it to the last saved commit.
- `git check-ignore -v <filepath>`
 - Shows why a file is being ignored by .gitignore rules.



THIS IS GIT. IT TRACKS COLLABORATIVE WORK
ON PROJECTS THROUGH A BEAUTIFUL
DISTRIBUTED GRAPH THEORY TREE MODEL.

COOL. HOW DO WE USE IT?

NO IDEA. JUST MEMORIZE THESE SHELL
COMMANDS AND TYPE THEM TO SYNC UP.
IF YOU GET ERRORS, SAVE YOUR WORK
ELSEWHERE, DELETE THE PROJECT,
AND DOWNLOAD A FRESH COPY.



[xkcd: Git](#)

How do I get started?

- In this week's mandatory assignment, you will create your (first?) GitHub repository
- VS Code allows you to manage your repositories well without having to memorise all the git commands
- For the deliverables (two weeks projects starting next week) you will work in pairs and use version control



[xkcd: Git](#)

Setting up a GitHub repository

- Easiest to create the repository on the web site and then clone it to your computer
- Chose between a Public or Private repository
 - When you hand in repos as part of an assignment, it must be public for us to be able to access it and grade your assignment
- Include a README.md that explains what the repository is for
- Include a .gitignore for files and directories that should not be tracked
 - GitHub has a set of default .gitignore for different programming languages
 - gitignore.io creates recommended .gitignore files for your specifications (for example: Python+VSCode+Windows)
- We recommend starting a new repository for each new project
 - Just as with virtual environments

A note on venv and git

- The virtual environment is usually mentioned in `.gitignore`, and thus not tracked
 - They are platform and OS dependent
 - They include hard-coded paths
 - The `.venv` directory is large and contains lots of files
 - Potential security risk
- Instead, include the recipe for your virtual environment
 - Create requirements file in terminal using pip: `python -m pip freeze > requirements.txt`
 - This writes all package name and versions to the file `requirements.txt`
 - This file can be used to build an equivalent venv on another machine: `python -m pip install -r requirements.txt`



**More about collaborative workflows
next week...**

Common file formats

- CSV (comma-separated value) files (.csv)
 - Very common format for storing data
 - Note: There should be no space around the commas separating the values
 - Note: Can use other separators than comma (semicolon, space, tab)
- Excel (and similar spreadsheets) (.xls, .xlsx, .ods)
 - pandas is a very useful library for working with spreadsheet style data

```
study_id,patient_id,date_of_birth,surgery_date,psa_value,psa_date
C08,92,1950-02-23,2006-09-02,0.02,2008-09-01
B41,106,1958-06-08,2009-02-01,0.01,2009-04-02
C08,14,1944-10-18,2006-07-21,0.03,2009-07-20
B41,118,1937-05-22,2006-12-02,0.03,2009-12-01
B41,9,1954-09-13,2005-03-01,0.04,2012-02-29
C08,41,1938-06-08,2007-07-04,0.05,2007-11-02
B41,49,1940-11-08,2007-05-21,0.04,2024-05-20
A12,91,1955-07-09,2007-11-28,0.05,2009-11-27
C08,68,1942-10-31,2006-05-14,0.01,2008-05-13|
B41,60,1953-05-16,2008-07-01,0.03,2011-07-01
B41,71,1960-05-09,2006-01-11,0.16,2008-01-10
```

Common file formats

- CSV (comma-separated value) files (.csv)
 - Very common format for storing data
 - Note: There should be no space around the commas separating the values
 - Note: Can use other separators than comma (semicolon, space, tab)
- Excel (and similar spreadsheets) (.xls, .xlsx, .ods)
 - pandas is a very useful library for working with spreadsheet style data
- Json ([JavaScript Object Notation](#)) (.json)
 - Much used for data exchange between client and server applications
 - Example: Jupyter Notebooks are JSON files (.ipynb)
 - Looks a lot like Python lists and dictionaries
 - The complete JSON grammar is described here: [JSON](#)

```
{  
  "editor.formatOnSave": false,  
  "editor.defaultFormatter": "charliermarsh.ruff",  
  "python.analysis.typeCheckingMode": "off",  
  "editor.inlineSuggest.enabled": false,  
  "editor.suggest.showSnippets": false  
}
```

Common file formats

- YAML ([YAML ain't Markup Language](#)) (.yaml)
 - Much used for configuration files
 - Can be used to store data in general
 - Has a similar indented block structure as Python code
 - Very human-readable (and human-writable) format
 - See also [YAML Ain't Markup Language \(YAML™\) Version 1.2](#) and pyyaml.org/wiki/PyYAMLDocumentation
- XML ([Extensible Markup Language](#)) (.xml)
 - Has been around for a long time, related to HTML
 - Very powerful
 - Verbose and complex and used less in data science contexts today
 - We will not look at reading it here

```
- Age: 24
  Name: Per
  Phone number: '12345678'
- Age: 23
  Name: Kari
  Phone number: '23456789'
```

```
<?xml version="1.0" encoding="UTF-8"?>
<library>
  <book category="fiction">
    <title>The Great Gatsby</title>
    <author>F. Scott Fitzgerald</author>
    <year>1925</year>
    <price currency="USD">14.99</price>
  </book>
</library>
```

Common file formats

- TOML ([Tom's Obvious, Minimal Language](#), originally: Tom's Own Markup Language) (.toml)
 - Much used for configuration files
 - In Python we often include a pyproject.toml
 - Designed to be easy to read and write

```
[project]
name = "inf201-h26"
version = "0.1.0"
requires-python = ">=3.11"

[tool.ruff]
target-version = "py314"
line-length = 120
```